

01: Getting Started Exercises

Introduction

In these exercises, you will:

- Confirm opencode is installed and connected to the `gpt-4.1-mini` model
- Ask opencode a basic question about this repository
- Generate the project's `AGENTS.md` file with `/init`
- Get a first look at `opencode.json`, opencode's configuration file

Prerequisites

1. Confirm opencode is installed, either in CLI:

```
opencode --version
```

Or open the Desktop app, choose New project, navigate to the exercise repo. 2. Confirm you're connected to a provider. If you haven't signed in yet, run opencode and use: `text /model` For this workshop your instructor will tell you which provider + model you can use (this repo's exercises are written for `gpt-4.1-mini`, the model made available ahead of this workshop). 3. Ensure you have the `atlas-agentic-demo` repo (this document's repo) downloaded, and then, from the repository root, confirm the project builds before you start: `mvn clean verify`

Exercises

Exercise 1.1: Ask a basic question

`opencode` has no separate "ask-only" chat panel the way M465 Copilot/GitHub Copilot does. Every session starts in **Build**, the default primary agent, which can read, edit, and run commands. You will restrict what it's allowed to do using permissions later. For now, just ask a question.

1. Open up either OpenCode Desktop app or launch `opencode` CLI from the repository root:

```
opencode
```

If on the desktop app, create a new project in the `atlas-agentic-demo` repo folder, and create a new session. 2. Type the following prompt and press enter: `text Tell me about the application in this repository. What does it do, and what technology stack does it use?` 3. Read the answer. Confirm it mentions

- the JDK `HttpServer`,
- SQLite,
- the five classes under `src/main/java/com/atlas/inventory/`
 - `Main`
 - `InventoryItem`
 - `InventoryRepository`
 - `InventoryService`
 - `InventoryServer`

Exercise 1.2: Generate AGENTS.md with /init

`AGENTS.md` is `opencode`'s persistent project guidance file. It is the equivalent of a custom instructions file for other harnesses (e.g. `CLAUDE.md`, `GEMINI.md`). Every later exercise assumes it exists and that `opencode` has read it.

1. In the same `opencode` `session`, run:

```
/init
```

2. `/init` scans the repository (`pom.xml` , `README.md` , the `src/` layout, existing tests) and creates the `AGENTS.md` file in the project root. It may ask you one or two clarifying questions first, just answer them.
3. Once it finishes, open the generated `AGENTS.md` file and read it. Confirm it captures at least:
 - the Maven build/test commands (for example `mvn clean verify` , `mvn test -Dtest=<ClassName>`),
 - the project structure (`InventoryItem` , `InventoryRepository` , `InventoryService` , `InventoryServer`),
 - any conventions it noticed (record types, no framework/ORM, JDK `HttpServer`).
4. Commit this file:

```
git add AGENTS.md
git commit -m "Add AGENTS.md via opencode /init"
```

You'll extend this file with your own rules in a later exercise.

Exercise 1.3: Re-run `/init` after making a change

`/init` is safe to re-run. If/when `AGENTS.md` already exists, it updates that file rather than starting from scratch. Exactly *how much* it rewrites depends on the model, so treat this as an observation exercise, not a pass/fail check.

1. Manually add a throwaway line to the bottom of `AGENTS.md` , for example `<!-- test line -->` .
2. Commit or stash nothing — just run `/init` again.
3. Run `git diff AGENTS.md` and read it. What did `/init` keep, what did it rewrite, and did your throwaway line survive?

There is no single correct answer here. The point is that model outputs are stochastic, not deterministic. Hence, `/init` is not idempotent and not guaranteed to preserve your hand-edits. so once you start maintaining `AGENTS.md` yourself, re-running `/init` is something you do deliberately and then review as a diff, not something you fire off casually.

Exercise 1.4: A first look at `opencode.json`

`AGENTS.md` tells opencode about *your project*. `opencode.json` configures *opencode itself*: models, providers, permissions, tools, keybinds, themes, and similar operational settings. It's a plain JSON file, and unlike `AGENTS.md` it isn't generated by a slash command, you create and edit it by hand.

1. From the repository root, look at the `opencode.json` that already ships with this workshop repo:

```
cat opencode.json
```

2. Note where `opencode` looks for config, and that it merges everything it finds rather than picking just one file:

- **Project config:** `opencode.json` in the repository root (what this exercise uses).
- **Global config:** `~/.config/opencode/opencode.json` applies across all your projects.
- Configs are **merged, not replaced**: a project `opencode.json` only overrides the specific keys it sets; everything else falls back to your global config or opencode's defaults.

3. Here is the most minimal `opencode.json` you could write:

```
{
  "$schema": "https://opencode.ai/config.json"
}
```

That's a completely valid config, as `opencode` falls back to defaults for everything: whichever model you last connected with, no server port override, and its built-in default permissions (Build can edit and run commands freely; nothing is pre-denied).

This repo's actual `opencode.json` is far from minimal. Read through it and confirm you can explain each section:

- `"$schema"` — gives you autocomplete/validation in editors that support JSON schema. Always present.
- There is no `"model"` entry in this file, so sessions use the model set in your global config. The workshop target is the `gpt-4.1-mini` model described above.
- `"autoupdate": true` — grants `opencode` permission to update itself in the background.
- `"server": { "port": 4096 }` — fixes the local server port instead of letting opencode pick one.
- `"permission"` — the biggest section, and the reason this file isn't minimal:
 - `"read"` allows reading everything (`"*": "allow"`) except an explicit denylist of secret-shaped paths (`*.env*`, `*id_rsa*`, `*.pem`, `*.key`, `*credentials*`).
 - `"edit": "ask"` means every file edit requires your approval — Build never edits silently in this repo, regardless of what mode you're in.
 - `"bash"` is a default of `"ask"` with a short allowlist of safe, read-only commands (`git status*`, `git diff*`, `git log*`, `ls*`, `pwd*`, `cat*`, `grep *`, and the test runners `mvn test*` / `mvn clean verify*`) and an explicit denylist of destructive ones (`rm *`, `rmdir *`, `git push*`, `git reset --hard*`, `git clean*`, `sudo *`, `chmod *`, ...) that are always blocked, never merely asked about.
 - **The same block covers PowerShell.** There is no separate `"powershell"` permission — opencode gates every shell command through `"bash"`, matching the rule against the command text, so on Windows the cmdlets are spelled out too. Hence the read-only allowances `Get-ChildItem*`, `Get-Content*`, `Select-String*`, `dir*`, `type *`, and the denials `*Remove-Item*`, `*Clear-Content*`, `del *`, `erase *`, `rd *`, `icacls *`, `takeown *`, `*Set-ExecutionPolicy*`, `*Invoke-Expression*`, `*-Verb RunAs*`. (The aliases `rm`, `ls`, `cat` and `pwd` resolve in PowerShell too, so the Unix-looking rules already cover those spellings.)

- Everything not on either list falls through to "ask" — including `find *`, which stays on `ask` deliberately. Otherwise, `find . -name '*.tmp' -delete` and `find . -exec rm {} \;` would slip straight past the `rm *` deny rule.

Two details make this file work, and both are worth knowing before you write your own:

- **Rules are matched against the whole command string, and the *last* matching rule wins.** That's why `"*": "ask"` sits at the top and the denials sit at the bottom: a later `deny` overrides an earlier `allow`. Reorder the file and you change its meaning.
- **A pattern like `rm *` only matches a command that starts with `rm`.** So `ls | xargs rm -rf` would sail through on the `ls*` allowance unless something catches it — which is what the `*| xargs rm*`, `*; rm *`, `*&& rm *` and `*| sh` rules at the end of the block are for. The PowerShell denials use the unanchored `*Remove-Item*` form for the same reason: it catches `Get-ChildItem -Recurse | Remove-Item -Force`, not just a bare `Remove-Item`.

A denylist only ever catches the spellings you thought of. That is the real reason the default is `ask` rather than `allow` — the deny rules are a safety net under the default, not a substitute for it.

- `"webfetch": "ask"` and `"external_directory": "ask"` gate network access and access outside the project directory the same way.
4. This is the config doing real work: it's what makes it safe to hand Build full tool access in these exercises without worrying about a stray `rm -rf`, a push, or a secret file being read into context.

You won't need to edit this file yourself in the fundamentals exercises. Permissions and other `opencode.json` settings are covered further as they come up (for example, Plan mode's default permissions in [02-context-modes-and-mentions.md](#)). This exercise is about knowing the file exists, what it's for, where it lives, and why the workshop's version looks the way it does compared to the minimal case above.